Service Oriented Computing and Applications (2026) 20:161–174
https://doi.org/10.1007/s11761-025-00476-5

ORIGINAL RESEARCH PAPER

Dynamic Multi-Objective Service Resource Scheduling via LLM-Optimized Fuzzy State Fusion and Reinforcement Learning Closed Loop

Zhengzuo Li[1] · Dianhui Chu[1] · Zhiying Tu[1] · Xin Hu[1] · Deqiong Ding[1]

Received: 29 May 2025 / Accepted: 29 September 2025 / Published online: 10 October 2025

Abstract
Efficient resource scheduling is a cornerstone for optimizing service quality and operational efficiency in modern healthcare, logistics, and cloud computing systems. However, existing methods often struggle with dynamic environments, multi-objective trade-offs, and stringent Service Level Agreement (SLA) constraints. To address these challenges, this paper proposes a novel framework named LFRL-MOS (LLM-Fuzzy-Reinforcement Learning Multi-Objective Scheduling). The framework integrates three key innovations: (1) LLM (Large Language Model) -enhanced fuzzy state fusion, which unifies real-time and predictive states through dynamically adjusted membership functions; (2) SLA-constrained Multi-Objective Ant Lion Optimizer (MOALO), which incorporates dynamic Pareto frontier adjustments and cross-department penalty mechanisms to ensure low-violation solutions; and (3) a dual-loop Proximal Policy Optimization (PPO) mechanism, which adaptively tunes parameters across all components for robust performance. Extensive experiments on simulated medical resource scheduling scenarios demonstrate that LFRL-MOS significantly outperforms state-of-the-art baselines in terms of SLA violation rate, average response time, resource utilization efficiency, and cost effectiveness. Our findings highlight the potential of combining advanced machine learning techniques with evolutionary optimization for addressing complex, dynamic scheduling problems.

Keywords Resource Scheduling · Multi-Objective Optimization · Reinforcement Learning · LLM-enhanced Fuzzy Logic · SLA Constraints

1 Introduction

With the exponential growth in complexity of modern service systems, efficient resource scheduling has become a critical challenge for ensuring service quality and operational efficiency [1]. In domains like healthcare, logistics, and cloud computing, service resources exhibit multi-attribute heterogeneity (e.g., skill levels, spatiotemporal constraints) and dynamic volatility (e.g., demand surges), requiring trade-offs among competing objectives such as temporal efficiency, economic cost, and SLA compliance [2].

For example, medical resource scheduling involves coordinating heterogeneous resources like physicians, equipment, and hospital beds. The key challenge is accurately characterizing interactions among real-time states, predictive demand, and SLA to improve resource utilization and patient satisfaction [3].

Existing approaches can be categorized into two classes: traditional optimization methods (e.g., Integer Programming, Genetic Algorithms) work well in static scenarios but struggle with dynamic environments [4]. Data-driven methods using RL (Reinforcement Learning) and evolutionary algorithms adapt better to changing conditions [5], yet face challenges in handling multi-objective trade-offs, SLA violations, and state uncertainties [6].

✉ Zhengzuo Li
hitwhlzz@163.com

Dianhui Chu
chudh@hit.edu.cn

Zhiying Tu
tzy_hit@hit.edu.cn

Xin Hu
hithuxin@hit.edu.cn

Deqiong Ding
mathddq@hit.edu.cn

[1] School of Computer Science and Technology, Harbin Institute of Technology (Weihai), No.2, West Wenhua Road, Weihai 264209, Shandong, China

Conventional methods have three limitations: 1) Decoupling real-time monitoring from predictive state evaluation leads to incomplete system characterization; 2) Static parametric models fail to adapt to non-convex objective spaces induced by resource sharing; 3) Expert-predefined parameters in fuzzy systems degrade solution quality over time.

Several studies explore advanced scheduling paradigms. For instance, Abraham et al. [7] review RL applications in cloud resource scheduling, emphasizing the need for sophisticated models. Zhang et al. [8] propose a DRL (Deep Reinforcement Learning) -based approach for industrial networks, showing superiority in handling complex task flows. Despite progress, limitations remain, such as slow convergence, reliance on static weight schemes, and insufficient SLA handling [9].

This paper addresses the following challenges:

- State representation fragmentation: Separating real-time monitoring from predictive evaluation results in incomplete system characterization.
- Multi-objective conflict escalation: Cross-unit allocation exacerbates SLA penalties, making the objective space non-convex.
- Parametric rigidity: Fixed parameters in fuzzy systems lead to declining solution quality over time.

To address these, we propose LFRL-MOS, integrating:

1. LLM-enhanced fuzzy state fusion: Maps real-time and predictive states into a unified semantic space via dynamic membership functions optimized by LLMs.
2. SLA-constrained Multi-Objective Optimization: Constructs a high-dimensional objective space with an enhanced MOALO, dynamically adjusting trap radius for low-violation solutions and employing Pareto optimization for trade-offs.
3. Reinforcement Learning-driven Dynamic Parameter Optimization: A dual-loop PPO mechanism updates fuzzy fusion, penalty terms, and MOALO parameters in real-time, forming a closed-loop “decision-feedback-optimization" chain.

The rest of this paper is organized as follows: Section 2 reviews existing resource scheduling methods. Section 3 defines the problem and sets the foundation for our approach. Section 4 details the proposed framework’s architecture and components. Section 5 evaluates the method’s performance through experiments. Finally, Section 6 concludes the paper and discusses future work.

2 Related Work

Resource scheduling is a core challenge in modern service systems, attracting significant attention from researchers. This section reviews existing methods, dynamic innovations, and key challenges, identifying gaps addressed by our work.

2.1 Resource Scheduling Methods

2.1.1 Traditional Optimization Methods

Traditional optimization methods, such as Integer Programming (IP), model scheduling problems using linear constraints and objective functions, performing well in static scenarios but struggling with dynamic demands and SLA constraints [10, 11]. Genetic Algorithms (GA), inspired by natural selection, excel in solving multi-objective problems through heuristic rule-based strategies [12, 13], yet often exhibit slow convergence in non-stationary environments and risk local optima [14]. Reinforcement learning (RL) applications in cloud resource scheduling highlight the limitations of traditional GAs and IPs in dynamic settings [7].

2.1.2 Dynamic Scheduling Innovations

Data-driven techniques powered by reinforcement learning (e.g., Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO)) demonstrate superior performance in handling complex task flows [8]. Dual-loop RL tunes parameters hierarchically, while Multi-Objective Ant Lion Optimizer (MOALO) combines evolutionary search with predator-prey simulation for effective multi-objective optimization [6, 15]. Co-evolutionary NSGA-III (Non-dominated Sorting Genetic Algorithm III) integrated with deep RL (CEGA-DRL) balances multiple objectives dynamically [16].

2.2 Challenges in Scheduling

2.2.1 State Representation

Accurate state representation is critical. Traditional methods focus on single-modal monitoring, neglecting predictive modeling’s value [17, 18]. Integrating real-time and predictive models remains challenging due to heterogeneous data sources [19]. Multi-objective interval type-2 fuzzy linear programming (MOIT2FLP) addresses uncertainties via membership functions [20].

2.2.2 Fuzzy Logic Systems

Fuzzy logic manages uncertainties effectively. Type-2 fuzzy systems handle parameter uncertainties and dynamic changes better than their counterparts through secondary member-

ships [21]. However, they rely heavily on expert-defined rules, which may become outdated. Large Language Models (LLMs) automate fuzzy rule generation, enabling continuous adaptation [22]. For instance, GP-CEA (Genetic Programming based Cooperative Evolutionary Algorithm) generates dispatching rules dynamically, enhancing exploration and exploitation capabilities [12].

2.2.3 SLA Constraints

Enforcing SLA constraints is central to resource scheduling. Static weight allocation schemes fail to adapt to evolving user requirements in dynamic environments [17, 19]. Dynamic adjustment methods, such as RPSO (Random Particle Swarm Optimization), DEEBS (Dual Experience-pool Elite Backtracking Strategy), and ϵ-IMOEA/D-M2M (Multi-Objective Evolutionary Aglorithms), improve performance under SLA constraints by combining niche techniques and ϵ-domination strategies [6].

2.3 Comparative Analysis and Inspiration

Traditional methods succeed in single-objective optimization but struggle with dynamic multi-objective scheduling. Evolutionary algorithms (e.g., NSGA-II (Non-dominated Sorting Genetic Algorithm II), ϵ-MOEA/D) perform well in low-dimensional regular Pareto frontier problems but falter in high-dimensional irregular fronts [9]. Most RL approaches influence specific parameters (e.g., policy updates or reward functions) without fully exploiting global optimization capabilities [12]. Static schemes lack flexibility, limiting adaptability in dynamic environments [17]. Our framework integrates LLM-enhanced fuzzy fusion, SLA-constrained MOALO, and dual-loop PPO to address these limitations dynamically.

3 Problem Statement

The core challenge in resource scheduling is optimally matching dynamic demands with multi-attribute resources, especially in complex systems like healthcare. This involves three key tensions:

1. Balancing temporal criticality (e.g., emergency response) with economic efficiency (e.g., maintenance costs).
2. Resolving conflicts between static allocation (e.g., specialist schedules) and dynamic demands (e.g., unplanned emergencies).
3. Addressing the duality of global optimization (e.g., hospital-wide balance) and local constraints (e.g., inter-department penalties).

Table 1 Notations Summary

Symbol	Definition
LFRL-MOS	Our Method.
LLM	Large Language Model.
SLA	Service Level Agreement.
MOALO	Multi-Objective Ant Lion Optimizer.
PPO	Proximal Policy Optimization.
$\mathcal{R}$	Enhanced resource state matrix
r_{ij}^{real}	Real-time availability variable
r_{ij}^{pred}	Predicted availability probability
a_{ij}	Attribute vector
J_k	Dynamic task request
t_{arrival}^k	Task arrival time of the k-th task
t_{deadline}^k	Deadline of the k-th task
D_k	Resource demand vector of the k-th task
SLA_k	SLA constraint tuple for the k-th task
ϕ_k	Urgency quantification value
C_k	Constraint dictionary for the k-th task
f_{time}	Time efficiency objective function
f_{cost}	Cost efficiency objective function
f_{util}	Utilization efficiency objective function
$P(X)$	Soft constraint penalty function
$\lambda_1, \lambda_2, \lambda_3$	Soft constraint penalty coefficients
C_{cross}	Cross-departmental cost coefficient
$\mathcal{F}_{\text{dyn}}$	Dynamic fuzzy state fusion module
$\mathcal{M}_{\text{SLA}}$	SLA-constrained multi-objective ant lion
$\mathcal{P}_{\text{dual}}$	Dual-loop PPO parameter tuner
$\mu_{\text{cost}}(x)$	Real-time cost membership function
$\mu_{\text{quality}}(r)$	Real-time quality membership function
$\mu_{\text{util}}(u)$	Real-time utilization membership function
$\mu_{\text{pred\_cost}}(x)$	Predicted cost membership function
$\mu_{\text{risk}}(x)$	Predicted risk membership function
$\mu_{\text{pred\_util}}(x)$	Predicted utilization membership function
R	Real-time fuzzy state set
P	Predicted fuzzy state set
$W(t)$	Dynamic weight function
r_{trap}	Dynamic trap radius in MOALO
r_0	Initial trap radius in MOALO
β	Dynamic trap radius coefficient
L_k	Cumulative SLA violation value
p_e	Elite preservation ratio

The main symbols used in this paper are listed in the Table. 1.

3.1 Formalization of Resource Scheduling

Definition 1 (Enhanced Resource State Matrix) The state of resources is represented as an enhanced matrix:

$$\mathcal{R} = \left[\langle r_{ij}^{\text{real}}, r_{ij}^{\text{pred}}, \mathbf{a}_{ij} \rangle \right]_{m \times T}, \tag{1}$$

Where:

- m: Number of resource types (e.g., ICU beds).
- T: Total time windows.
- $r_{ij}^{\text{real}} \in \{0, 1\}$: Real-time availability.
- $r_{ij}^{\text{pred}} \in \{0, 1\}$: Predicted availability.
- $\mathbf{a}_{ij} \in \mathbb{R}^p$: Attribute vector including type code, specialty tags, utilization rate, and SLA tuple.

Definition 2 (Dynamic Task Request) Each task is modeled as:

$$J_k = \left(t_k^{\text{arrival}}, t_k^{\text{deadline}}, \mathbf{D}_k, \mathbf{SLA}_k, \phi_k, \mathbf{C}_k \right), \tag{2}$$

Where:

- t_k^{arrival}: Arrival timestamp.
- t_k^{deadline}: Deadline.
- $\mathbf{D}_k$: Resource demand vector.
- $\mathbf{SLA}_k = (\tau_k^{\max}, c_k^{\max}, q_k^{\min})$: SLA constraints (response time, cost, quality).
- ϕ_k: Urgency level.
- $\mathbf{C}_k$: Constraints dictionary.

3.2 Multi-Objective Optimization

Resource scheduling aims to find Pareto optimal solutions for conflicting objectives:

$$\min \mathbf{f}(\mathbf{x}) = [f_{time}, f_{cost}, f_{util}]^T, \tag{3}$$

Where:

- f_{time}: Weighted response time.
- f_{cost}: Cost efficiency.
- f_{util}: Resource utilization balance.

3.3 Constraints

In a medical scenario, constraints are defined as:

- Hard Constraints:
  - Mutual exclusion: $\sum_{j=1}^{T} x_{ij} \leq 1, \forall i$.
  - Qualification: $\texttt{specialty}(i) \sqsubseteq \mathbf{D}_k$

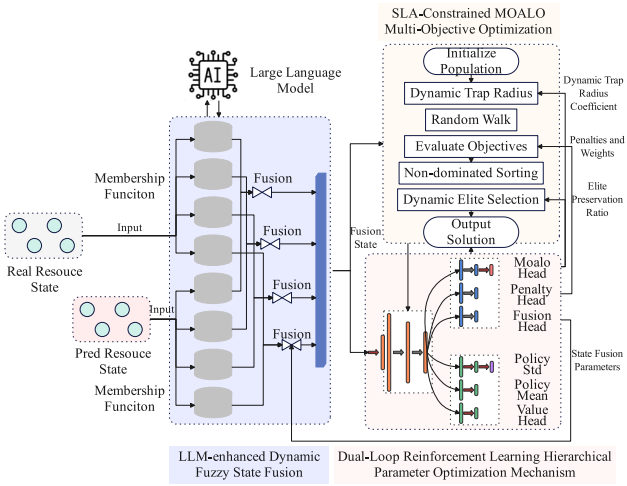

Fig. 1 The framework of LFRL-MOS

- Soft Constraints:

$$P(\mathbf{X}) = \lambda_1 C_{cross} + \lambda_2 \sum_{h=1}^{H} \max(0, w_h - 12) + \lambda_3 \sum_{k=1}^{K} \max(0, \Delta t - (t_{k+1} - t_k)), \tag{4}$$

Where:

- $\texttt{specialty}(i)$: Expertise tags for resource i.
- w_h: Cumulative working hours.
- $w_{\max}$: Maximum daily working hours.
- $\Delta t_{\min}$: Minimum cooldown duration.
- $\lambda_1, \lambda_2, \lambda_3$: Penalty coefficients.

4 Methodology

4.1 Overall Architecture

This study proposes a coordinated multi-objective optimization algorithm integrating LLM-enhanced fuzzy theory, SLA-constrained MOALO, and reinforcement learning to address decision-making challenges between dynamic real-time demands and service resources. The algorithm features three core components: (1) LLM-enhanced dynamic fuzzy state fusion, (2) SLA-constrained MOALO multi-objective optimization, and (3) a dual-loop PPO parameter tuner, forming a closed-loop collaborative optimization framework. Its key innovation lies in the synergistic mechanism between fuzzy fusion of real-time/predictive resource states and multi-objective optimization, as illustrated in Fig. 1.

$$\Psi = \langle \mathcal{F}_{\text{dyn}}, \mathcal{M}_{\text{SLA}}, \mathcal{P}_{\text{dual}} \rangle, \tag{5}$$

where:

- $\mathcal{F}_{\text{dyn}}$: LLM-enhanced dynamic fuzzy state fusion module
- $\mathcal{M}_{\text{SLA}}$: SLA-constrained multi-objective ant lion optimizer
- $\mathcal{P}_{\text{dual}}$: Dual-loop PPO parameter tuner

1. **LLM-enhanced Dynamic Fuzzy State Fusion**: This module integrates real-time and predicted states into a unified semantic space using LLM-generated fuzzy rules. It dynamically adjusts membership functions and fusion weights via inner-loop PPO, ensuring robust performance during demand surges.
2. **SLA-Constrained MOALO**: The enhanced MOALO optimizer incorporates SLA constraints with three key features: dynamic Pareto frontier adjustment, cross-department penalty terms, and equilibrium optimization balancing temporal responsiveness, cost, resource utilization, and SLA compliance.
3. **Dual-Loop PPO**: This system adapts parameters across all components. The inner loop tunes fuzzy inference, SLA penalties, and MOALO hyperparameters in real-time, while the outer loop optimizes the PPO policy through gradient-based exploration. This bi-level architecture ensures precision and avoids convergence issues of heuristic methods.

4.2 LLM-enhanced Dynamic Fuzzy State Fusion

4.2.1 Membership Function Design

To address the delayed response of static membership functions in dynamic scenarios, this study designs dynamic membership functions for real-time and predictive states.

1. **Real-Time Cost Modeling**: Expenditure follows a Gaussian distribution. The membership function is:

$$\mu_{\text{cost}}(x) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right), \tag{6}$$

where x: actual cost, c_t: mean cost, σ: standard deviation (updated via LLM).

2. **Real-Time Quality Modeling**: A resource-tiered quality framework uses discrete step functions. For physicians:

$$\mu_{\text{quality}}(r) = \begin{cases} a_1 & \phi(r) \le \theta_1 \\ a_2 & \theta_1 < \phi(r) \le \theta_2 \\ a_3 & \text{otherwise} \end{cases}, \tag{7}$$

where $\phi(r)$: quality score based on certification.

3. **Real-Time Utilization Modeling**: Utilization is captured using a linear membership function:

$$\mu_{\text{util}}(u) = u, \tag{8}$$

4. **Predictive Cost Modeling**: Predictive cost uncertainty is modeled using a Gaussian function:

$$\mu_{\text{pred\_cost}}(x) = \exp\left(-\frac{(x-\hat{c}_t)^2}{2\sigma_t^2}\right), \tag{9}$$

5. **Predictive Risk Modeling**: Operational risks are modeled using an exponential function:

$$\mu_{\text{risk}} = 1 - e^{-kx}, \tag{10}$$

6. **Predictive Utilization Modeling**: Forecasting balances precision and robustness via a triangular function:

$$\mu_{\text{pred\_util}}(x) = \max\left(0, 1 - \frac{|x-\hat{u}_t|}{w_t}\right), \tag{11}$$

where $\hat{u}_t$: forecast utilization, w_t: error bound (both LLM-refined).

4.2.2 LLM-Optimized Fuzzy Parameter Configuration

To address the limitations of traditional fuzzy systems relying on static expert knowledge, this study introduces an LLM-driven fuzzy parameter optimization framework. It uses a pre-trained large language models as virtual domain experts to dynamically generate and refine fuzzy rules based on real-time operational contexts. The system is built on the DeepSeek-V3 architecture, a 67.1-billion-parameter model with a 64K context window [23]. Interaction with the model follows a zero-shot inference protocol via API, and crucially, no fine-tuning is performed to maintain model generality and broad applicability. Inputs to the model consist of real-time resource states encoded into structured textual prompts, while outputs are generated in JSON format, specifying optimization parameters as outlined in Appendix A. Domain-specific knowledge is integrated through designed prompt engineering, ensuring context-aware reasoning without altering the underlying model weights. This deliberate avoidance of fine-tuning supports the preservation of the model's generalization capabilities while enabling effective, dynamic decision-making in the healthcare operations context.

In order to enhance the efficiency of model calls and reduce the likelihood of failures, a precision cache mechanism is implemented. Whenever a request for parameter optimization is made, the system first checks if the corresponding state vector $S_t = [\mu_{cost}, \mu_{quality}, \mu_{util}, \mu_{pred\_cost}, \mu_{risk}, \mu_{pred\_util}]$ exists within the cache. If an exact match is found,

the cached response is returned almost instantaneously, typically within less than 1 millisecond. For cases where there is no matching entry (cache miss), the system proceeds to query the DeepSeek API asynchronously, with a timeout limit set to 500 milliseconds. Upon receiving a successful response, the system updates the cache using a Least Recently Used (LRU) strategy, thereby ensuring that the most relevant entries are retained for future use. In the event of a timeout or any other exception during the API call, the system employs fallback mechanisms: it returns the historical best-known result, making it well-suited for real-time applications in dynamic environments.

This approach enables real-time adaptation of control parameters using current and predicted states, unlike static systems. By integrating historical analysis, it provides data-driven insights for robust tuning. Its modular design ensures scalability for varying complexities and resource constraints, improving system responsiveness and efficiency. The inclusion of the cache mechanism further enhances these qualities by reducing latency and increasing the robustness of responses, thus supporting more agile and reliable decision-making processes.

4.2.3 Dynamic Fuzzy State Fusion

To integrate real-time and predictive resource states, we propose a tri-modal fuzzy fusion system that dynamically balances observed and projected conditions. The framework is defined as:

$$\mathcal{F} = \langle R, P, W \rangle, \tag{12}$$

where:

- $R = \{\mu_{\text{real}}(x) \mid x \in X\}$: Real-time state fuzzy set,
- $P = \{\mu_{\text{pred}}(x) \mid x \in X\}$: Predicted state fuzzy set,
- $W : [0, T] \times \mathbb{R}^+ \to [0, 1]$: Dynamic weight function.

The fused membership function satisfies:

$$\begin{aligned} &\min(\mu_{\text{real}}(x), \mu_{\text{pred}}(x)) \le \mu_{\text{fused}}(x) \\ &\quad \le \max(\mu_{\text{real}}(x), \mu_{\text{pred}}(x)), \end{aligned} \tag{13}$$

and is computed as:

$$\mu_{\text{fused}}(x) = \mathcal{W}(t) \cdot \mu_{\text{pred}}(x) + (1 - \mathcal{W}(t)) \cdot \mu_{\text{real}}(x), \tag{14}$$

where $\mathcal{W}(t)$ is learned via dual-loop reinforcement learning to adapt to dynamic uncertainties.

Table 2 Definitions of Symbols

Symbol	Description
x_{ijt}	Assigned binary variable
$z_{ijkk't}$	Cross-department scheduling indicator
cap_j	Maximum service capacity
Ω	Set of allowable user-resource pairings
I	User set
J	Resource set (doctors, equipment, etc.)
J_i^{pref}	Preferred resource set of user i
T	Set of discretized time windows
c_j^{base}	Base usage cost of resource j
$\alpha_{kk'}$	Cross-department penalty coefficient
$t_i^{arrival}$	Arrival time of user i
$t_i^{deadline}$	Latest start time for user i (SLA threshold)
c_i^{actual}	Actual allocation cost for user i
c_i^{budget}	Budget constraint for user i
w_i^t	SLA time violation weight for user i
w_i^c	SLA cost violation weight for user i
w_i^q	SLA quality violation weight for user i

4.3 SLA-Constrained MOALO Multi-Objective Optimization

4.3.1 Objective Function

To address conflicting objectives and SLA constraints in resource scheduling, we propose a Multi-Objective Ant Lion Optimizer (MOALO) framework that integrates inter-departmental borrowing while ensuring SLA compliance. The optimization problem is reformulated as:

$$\begin{aligned} \min \mathbf{f}(\mathbf{x}) &= [f_{time}, f_{cost}, f_{util}, f_{sla}] \\ \text{s.t.} \quad &\sum_i x_{ijt} \le \text{cap}_j, \forall j, t \\ &\sum_{j,t} x_{ijt} = 1, \forall i \\ &x_{ijt} \le \mathbb{I}((i, j) \in \Omega), \forall i, j, t, \end{aligned} \tag{15}$$

where x_{ijt}: binary assignment variable, $z_{ijkk't}$: cross-department indicator, and other symbols are defined in Table 2.

The objectives are:

1. Time efficiency (f_{time}) reflects the waiting duration from patient arrival to service provision:

$$f_{time} = \frac{1}{|I|} \sum_{i,j,t} x_{ijt}(t - t_i^{\text{arrival}}), \tag{16}$$

2. Cost efficiency (f_{cost}) includes the fixed costs of fundamental resource and the variable expenses associated with cross-departmental coordination.

$$f_{cost} = \sum_{i,j,t} c_j^{\text{base}} x_{ijt} + \sum_{i,j,k,k',t} c_j^{\text{base}} (1 + \alpha_{kk'}) z_{ijkk't}, \tag{17}$$

3. Resource utilization (f_{util}) is formulated in Eq. 18:

$$f_{util} = -\frac{1}{|J||T|} \sum_{j,t} \left(\frac{\sum_i x_{ijt}}{\text{cap}_j} \right)^2, \tag{18}$$

4. SLA constraint (f_{sla}) includes temporal violations, cost overruns, and quality breaches:

$$\begin{aligned} f_{sla} = & \frac{1}{|I|} \sum_i \Big[w_i^t \max(0, t_i^{\text{start}} - t_i^{\text{deadline}}) \\ & + w_i^c \max(0, c_i^{\text{actual}} - c_i^{\text{budget}}) \\ & + w_i^q \left(1 - \sum_{j \in J_i^{\text{pref}}} x_{ij} \right) \Big], \end{aligned} \tag{19}$$

Cross-department penalties $\alpha_{kk'}$ and SLA weights w_i^t, w_i^c, w_i^q are adaptively adjusted via inner-loop PPO.

4.3.2 SLA-Constrained MOALO

For the formulated multi-objective optimization framework, we propose an improved MOALO algorithm incorporating SLA constraints with two key features: a) A dynamic trapping radius mechanism, and b) An adaptive elite preservation strategy.

To address the issue of fixed trapping radii causing excessive local exploitation, this study proposes an SLA-driven dynamic trapping radius mechanism:

$$r_{\text{trap}}(t) = r_0 \left(1 + \beta \log \left(1 + \sum_{k=1}^{T} L_k \right) \right), \tag{20}$$

where r_0 denotes initial trap radius, L_k represents accumulated SLA violations. When L_k exceeds a threshold, r_{trap} expands logarithmically to encourage broader exploration. The scaling coefficient β is updated via inner-loop PPO.

For the elite retention strategy, the elite preservation ratio p_e is dynamically adjusted by PPO to increase when high violation rates are detected, balancing solution quality and diversity. Elite selection uses Pareto front ranking and crowding distance metrics for uniform distribution.

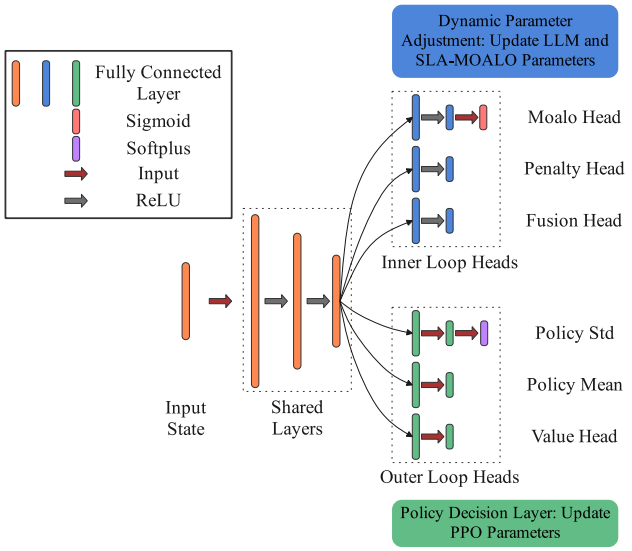

Fig. 2 Dual-Loop Reinforcement Learning Framework

4.3.3 Algorithm Flow

The enhanced MOALO approach integrates two core innovations: dynamic trapping radius and adaptive elite preservation. The detailed workflow is presented in Algorithm 1 in the Appendix and consists of three phases:

1. **Initialization**: Randomly generate N candidate solutions, each containing an allocation matrix and inter-department scheduling flags.

2. **Evolutionary Loop**: (1) Adjust search radius adaptively using PPO inner-loop updates. (2) Repair solutions by enforcing resource capacity and allocation integrity. (3) Perform non-dominated sorting with SLA constraints and dynamic elite retention optimized by PPO.

3. **Output**: Return the Pareto-optimal set, while hyperparameters are continuously adapted via PPO's performance-driven optimization.

4.4 Dual-Loop Reinforcement Learning Hierarchical Parameter Optimization Mechanism

To address adaptive optimization challenges in real-time/predicted state fusion, cross-department coordination, and SLA trade-offs, this paper proposes a dual-loop reinforcement learning framework for hierarchical parameter optimization. The architecture is shown in Fig. 2.

The model begins with a shared state encoding layer, where resource states and SLA penalties are normalized into high-dimensional feature vectors:

$$h_l = \text{ReLU}(W_l h_{l-1} + b_l), \quad l \in \{1, 2, 3\}. \tag{21}$$

where h_l denotes the hidden state at layer l. W_l shows the weight matrix for layer l. b_l is the bias vector for layer l.

Following the shared layer, the architecture branches into two parts:

1. Inner Loop (Dynamic Parameter Adjustment): Outputs state fusion parameters, cross-department penalties, MOALO hyperparameters, and state value $V_\theta(s_t)$, all normalized to [0, 1] via sigmoid activation.
2. Outer Loop (Policy Decision Layer): Outputs policy mean μ (tanh-activated) and variance σ (Softplus-constrained).

Notably, the inner loop's loss function combines value loss $V_\theta(s_t)$ with weighted parameter-specific losses. The outer loop's loss function decomposes into three components: policy loss, value function loss, and entropy regularization term.

$$\mathcal{L}_{\text{inner}} = \mathcal{L}_{\text{value}} + \mathcal{L}_{\text{parameter}}, \tag{22}$$

$$\mathcal{L}_{\text{outer}} = \mathcal{L}_{\text{policy}} + \mathcal{L}_{\text{value}} + \mathcal{H}(\pi), \tag{23}$$

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}_t\left[\min\left(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t\right)\right], \tag{24}$$

$$\mathcal{L}_{\text{value}} = \frac{1}{2}\mathbb{E}_t\left[\left(V_\theta(s_t) - V_{\text{target}}(s_t)\right)^2\right], \tag{25}$$

$$\begin{aligned}\mathcal{L}_{\text{entropy}} &= -\mathbb{E}_t\left[\mathcal{H}(\pi_\theta(\cdot|s_t))\right] \\ &= -\mathbb{E}_t\left[\log \pi_\theta(a_t|s_t) + \text{const}\right],\end{aligned} \tag{26}$$

Where:

- $\mathcal{L}_{\text{parameter}}$: Loss for state fusion and penalty parameters.
- $\mathcal{L}_{\text{policy}}$: Policy loss for expected returns.
- $\mathcal{L}_{\text{value}}$: Critic loss for value prediction.
- $\mathcal{L}_{\text{entropy}}$: Entropy term for exploration.
- $\mathbb{E}_t[\cdot]$: Expectation over time steps t.
- $r_t = \exp\left(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t)\right)$: Probability ratio.
- $A(s_t, a_t)$: Advantage function.
- $\text{clip}(x, 1-\epsilon, 1+\epsilon)$: Clipping function for stability.
- $V_\theta(s_t)$: Estimated state value.
- $V_{\text{target}}(s_t)$: Target state value.
- $\mathcal{H}(\pi_\theta(\cdot|s_t))$: Policy entropy for uncertainty.

The PPO-based dual-loop reinforcement learning framework optimizes the MOALO algorithm through hierarchical parameter adaptation, effectively handling dynamic state fusion under uncertainty while enforcing SLA constraints. The optimization results are fed back to the PPO controller, forming a closed-loop learning system.

5 Experiments

5.1 Experimental Setup

To systematically evaluate the effectiveness of our LFRL-MOS framework, we design experiments addressing three key research questions:

RQ1 (Model Effectiveness): How does LFRL-MOS perform compared to state-of-the-art baselines on simulated medical resource scheduling scenarios?

RQ2 (Component Analysis): What is the relative contribution of each core component (LLM-enhanced fuzzy fusion, SLA-constrained MOALO, and dual-loop RL optimization) to the overall performance?

RQ3 (Parameter Sensitivity) How do critical hyperparameters (e.g., MOALO trap radius r_0 and learning rate ratio in dual-loop RL) influence model performance?

RQ4 (Time Efficiency): What is the end-to-end latency and computational overhead of LFRL-MOS, and how does its performance (including LLM inference time) scale under increasing workload, demonstrating its practical feasibility for real-time applications?

5.2 Simulation Dataset and Baselines

We conduct comprehensive experiments using a simulated medical resource scheduling scenario that captures both temporal and spatial dynamics in healthcare systems. The simulation generates data for patient arrivals, resource demands, and SLA constraints.

The simulated data were generated using a parameterized discrete-event simulation framework grounded in real-world medical operations. Core data sources include medical resource configurations—such as physician specialty distribution, equipment quantities, and bed capacity—extracted directly from the publicly available information on the official website of a tertiary hospital. This simulation system has been employed in a previously published study, from which corresponding simulation data were obtained [24]. Furthermore, service pricing benchmarks are aligned with the Guangzhou's National Basic Medical Service Price Catalog (2024) in China to initialize cost parameters within the simulation model.

- Simulation Environment: Following the methodology in [25], patient arrivals are modeled as Poisson processes with varying arrival rates to reflect the stochastic nature of real-world healthcare demand. Resource availability (e.g., doctors and medical equipment) is subject to dynamic uncertainty and modeled using normal distributions to capture fluctuations in staffing and equipment access. Service Level Agreement (SLA) constraints—such as priorities and deadlines—are randomly assigned to tasks, simulating the heterogeneous and time-sensitive requirements commonly observed in clinical settings. This data generation mechanism ensures that the simulation closely aligns with real-world operational conditions.

- Dataset Characteristics: The simulation involves 10,000 patients over one month, divided into emergency cases and general outpatient visits. The system includes 2000 resource units distributed across departments. Emergency departments have priority access to idle resources from outpatient departments, ensuring rapid response without disrupting routine operations.

This setup allows for a comprehensive evaluation of the challenges faced in hospital resource management and provides a robust testbed for assessing the proposed resource scheduling algorithm.

For performance benchmarking, we carefully select four representative baseline methods spanning different paradigms:

1. NSGA-II (Non-dominated Sorting Genetic Algorithm II): Traditional multi-objective genetic algorithm [26].
2. NSGA-III (Non-dominated Sorting Genetic Algorithm III): Advanced version of NSGA-II for many-objective optimization [27].
3. FACO (Fuzzy-Ant Colony Optimization): Combines fuzzy logic with ant colony optimization for resource allocation [28].
4. CEGA-DRL (Co-Evolutionary NSGA-III combined with Deep Reinforcement Learning): Balancing multiple objectives under dynamic conditions [16].
5. SAQFA (SLA-aware Admission Control and Quantum-inspired Firefly Algorithm): A hybrid framework that integrates the SLA-aware Admission Control Heuristic (SACH) for intelligent task admission and a Quantum-inspired Firefly Algorithm (QFA) [29].

All baselines are adapted to incorporate SLA constraints and spatiotemporal features while preserving their original loss functions and optimization strategies.

All comparative algorithms (including NSGA-II, NSGA-III, FACO, CEGA-DRL, SAQFA and our proposed method) share identical initial weight settings for fair benchmarking. Specifically:

- **Initial weights:** $\mathbf{w} = [w_1, w_2, w_3, w_4] = [0.4, 0.3, 0.2, 0.1]$ for all objectives (Resource Utilization Efficiency, Average Response Time, Cost Efficiency, SLA Violation Rate).
- **Baseline methods:** Maintain fixed weights throughout optimization ($\Delta\mathbf{w} = 0$).
- **Our method:** Dynamically adjusts weights via the dual-Loop reinforcement learning hierarchical parameter optimization mechanism.

5.3 Evaluation Metrics

We employ a set of metrics commonly used in resource scheduling to comprehensively assess the quality and efficiency of the proposed framework:

1. **SLA Violation Rate** (VR): Measures the proportion of tasks violating SLA constraints.

$$VR = \frac{\text{Number of SLA-violated tasks}}{\text{Total number of tasks}},$$

2. **Average Response Time** (ART): Measures the average time taken to allocate resources.

$$ART = \frac{\sum_{i=1}^{N} t_i}{N},$$

3. **Resource Utilization Efficiency** (RUE): Measures the balanced utilization of resources.

$$RUE = 1 - \frac{\sigma(u_i)}{\bar{u}},$$

4. **Cost Efficiency** (CE): Measures the total operational cost normalized by the total number of tasks.

$$CE = \frac{\text{Total operational cost}}{\text{Total number of tasks}}.$$

All metrics are computed per-task and averaged across all test instances.

5.4 Environment and Hyperparameter Settings

The experimental environment was established using Python 3.10 with PyTorch 2.5 for simulation and optimization.

Key hyperparameters and configurations are summarized as follows:

- **LLM-Fuzzy Module**: Rule update frequency = 6h, temperature = 0.7.
- **MOALO Optimizer**: Initial trap radius $r_0 = 0.2$, initial elite retention ratio = 0.2 (adaptively tuned via PPO during optimization).
- **Dual-Loop RL**: Inner loop learning rate = 3×10^{-4}, outer loop learning rate = 1×10^{-4}, dropout probability = 0.2.

Training protocol employs the AdamW optimizer with an initial learning rate of 1×10^{-3} and batch size of 64. Regularization techniques include L2 weight decay (1×10^{-5}), early stopping with 20-epoch patience monitored on validation SLA Violation Rate (VR).

Table 3 Performance Comparison Across Methods and Data Scales

Method	Data Scale [1]	VR	ART, sec	RUE	CE
LFRL-MOS	100, 20	0.331	4.1	**0.79**	59.5
	1,000, 200	0.384	50.99	**0.783**	45.23
	5,000, 1,000	**0.473**	**225.876**	**0.778**	43.898
	10,000, 2,000	**0.697**	491.121	**0.758**	**46.331**
	50,000, 5,000	**0.984**	2398.318	**0.744**	**49.67**
NSGA-II	100, 20	0.147	2.3	0.709	**41.1**
	1,000, 200	0.413	49.96	0.681	42.0
	5,000, 1,000	0.689	256.28	0.681	42.0
	10,000, 2,000	0.982	498.84	0.622	50.254
	50,000, 5,000	0.993	2479.214	0.603	55.335
NSGA-III	100, 20	0.143	2.1	0.729	41.7
	1,000, 200	0.375	49.95	0.684	**40.12**
	5,000, 1,000	0.614	245.686	0.689	49.77
	10,000, 2,000	0.962	498.787	0.632	49.832
	50,000, 5,000	0.99	2473.783	0.611	54.489
FACO	100, 20	0.313	2.6	0.767	53.2
	1,000, 200	0.624	52.74	0.608	46.5
	5,000, 1,000	0.891	246.308	0.618	51.726
	10,000, 2,000	0.997	501.982	0.629	50.955
	50,000, 5,000	0.999	2499.551	0.583	59.313
CEGA-DRL	100, 20	**0.112**	2.1	0.785	50.5
	1,000, 200	**0.334**	51.82	0.776	41.05
	5,000, 1,000	0.497	244.642	0.712	**41.224**
	10,000, 2,000	0.772	497.784	0.688	46.491
	50,000, 5,000	0.988	2464.065	0.671	52.164
SAQFA	100, 20	0.135	**1.8**	0.764	45.58
	1,000, 200	0.35	**47.05**	0.743	42.741
	5,000, 1,000	0.497	237.99	0.717	44.979
	10,000, 2,000	0.806	**485.752**	0.669	50.462
	50,000, 5,000	0.985	**2377.53**	0.631	49.962

[1] The number of (Patients, Resources)

5.5 Results and Analysis

5.5.1 RQ1: Model Effectiveness

To systematically evaluate the performance of the LFRL-MOS framework across varying data scales, we conducted experiments comparing it against four state-of-the-art baselines: NSGA-II, NSGA-III, FACO, CEGA-DRL and SAQFA. The experimental scenarios were designed to simulate medical resource scheduling tasks with patient counts ranging from 100 to 50,000 and corresponding resource units ranging from 20 to 5,000. Key evaluation metrics include SLA Violation Rate (VR), Average Response Time (ART), Resource Utilization Efficiency (RUE), and Cost Efficiency (CE).

The detailed results are summarized in Table 3 below.

1. **SLA Violation Rate** (VR):
   - For small datasets (e.g., 100 patients, 20 resources), CEGA-DRL achieved the lowest $VR = 0.112$, followed closely by SAQFA (0.1345) and NSGA-III (0.143). LFRL-MOS had a higher $VR = 0.331$.
   - For large datasets (e.g., 50,000 patients, 5,000 resources), LFRL-MOS achieved the lowest $VR = 0.984$, slightly outperforming SAQFA (0.9849), CEGA-DRL (0.988), NSGA-III (0.99), and NSGA-II (0.993), and significantly better than FACO (0.999).
2. **Average Response Time** (ART):
   - On small datasets, SAQFA achieved the shortest $ART = 1.8$ seconds, outperforming CEGA-DRL (2.1) and LFRL-MOS (4.1).

- On large datasets, SAQFA performed best with $ART = 2377.53$ seconds, followed by LFRL-MOS (2398.318), both significantly faster than NSGA-II, NSGA-III, and FACO.

3. **Resource Utilization Efficiency** (RUE):
   - LFRL-MOS consistently achieved the highest RUE across all scales, e.g., 0.79 (small) and 0.744 (large). It outperformed SAQFA (0.631 at large scale), NSGA-II (0.603), FACO (0.583), and CEGA-DRL (0.671).
4. **Cost Efficiency** (CE):
   - For small datasets, LFRL-MOS showed the highest $CE = 59.5$, while SAQFA achieved 45.58, comparable to NSGA-II and NSGA-III.
   - For the largest dataset, LFRL-MOS achieved the best $CE = 49.67$, followed closely by SAQFA (49.9618), outperforming NSGA-II (55.335), NSGA-III (54.489), CEGA-DRL (52.164), and FACO (59.313).

From the experimental results, it is evident that LFRL-MOS offers significant advantages in solving dynamic multi-objective medical resource scheduling problems. Despite a higher SLA violation rate on small datasets, LFRL-MOS demonstrated superior performance on large-scale scenarios, achieving the lowest VR and highest RUE among all methods. Notably, the SAQFA method shows strong competitiveness, particularly in ART and VR, achieving the fastest response times and second-lowest violation rate at scale. However, LFRL-MOS maintains a clear edge in resource utilization and cost efficiency at large scales. The method effectively balanced trade-offs between VR, ART, RUE, and CE, particularly excelling in high-dimensional, complex scheduling environments.

5.5.2 RQ2: Component Analysis

To further validate the contributions of each core component in LFRL-MOS, we conducted an ablation study by systematically removing the LLM-enhanced fuzzy fusion module, the dual-loop reinforcement learning (RL) module, and the SLA-constrained MOALO module. Below are the detailed results and analyses see Table 4.

The ablation study shows that removing any core component significantly degrades LFRL-MOS's performance. The full model achieves $VR = 0.697$, $ART = 491.121$ s, $RUE = 0.758$, and $CE = 46.331$. In contrast, variants exhibit worse performance: $VR \geq 0.994$, $ART > 500$ s, $RUE < 0.623$, and unstable $CE > 46.331$.

Removing the LLM-enhanced fuzzy fusion module increases VR to 0.994, ART to 500.215 s, and decreases RUE to 0.616. This degradation occurs due to the loss of accurate state representation for dynamic environments.

Table 4 Performance Comparison Between Full Model and Variants

Method	VR	ART	RUE	CE
LFRL-MOS	**0.697**	**491.121**	**0.758**	**46.331**
Removed LLM	0.994	500.215	0.616	53.75
Removed RL	0.997	513.457	0.594	54.5
Removed MOALO	0.995	502.364	0.623	50.226

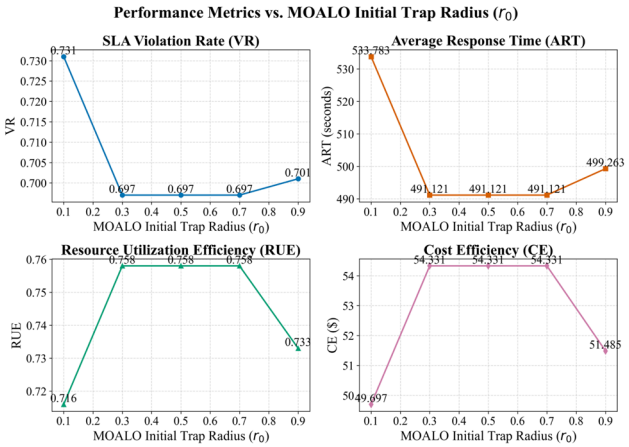

Fig. 3 Performance Metrics of Different MOALO Initial Trap Radius r_0

Eliminating the dual-loop reinforcement learning module causes the largest performance drop: $VR = 0.997$, $ART = 513.457$ s, and $RUE = 0.594$. Its absence prevents timely parameter updates, reducing responsiveness to high loads and demand surges.

Removing the SLA-constrained MOALO module results in $VR = 0.995$, $ART = 502.364$ s, $RUE = 0.623$, and $CE = 50.226$. While some metrics slightly improve, overall instability highlights its importance in balancing SLA constraints and multi-objective trade-offs.

These findings confirm the synergy among the LLM-enhanced fuzzy fusion, dual-loop reinforcement learning, and SLA-constrained MOALO modules, which ensures balanced optimization of SLA satisfaction, response time, resource utilization, and cost control.

5.5.3 RQ3: Parameter Sensitivity

To investigate the influence of key hyperparameters on LFRL-MOS performance, we analyzed the impact of MOALO's initial trap radius (r_0) and the inner-to-outer loop learning rate ratio. Below are the results.

1. **Impact of MOALO Initial Trap Radius** (r_0): The initial trap radius (r_0) balances exploration and exploitation.

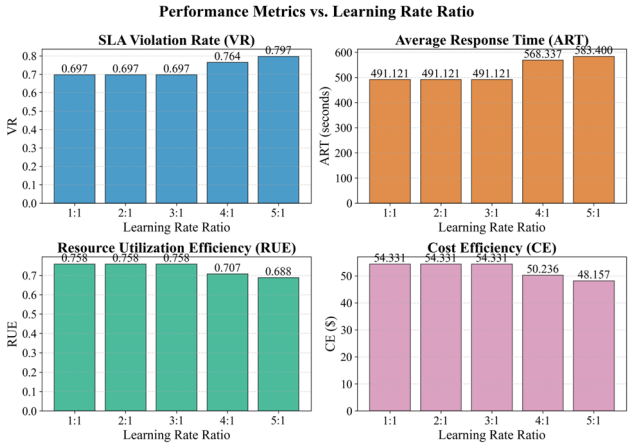

Fig. 4 Performance Metrics of Different Learning Rate Ratio

Testing $r_0 \in [0.1, 0.9]$, optimal performance is observed for $r_0 \in [0.3, 0.7]$. The result is shown in the Fig. 3.

- $VR = 0.697$ stabilizes within this range but rises to 0.701 at $r_0 = 0.9$.
- $ART = 491.121$ s degrades slightly to 499.263 s at $r_0 = 0.9$.
- $RUE = 0.758$ peaks for $r_0 \in [0.3, 0.7]$ but drops to 0.716 and 0.733 at $r_0 = 0.1$ and 0.9, respectively.
- $CE = 54.331$ is optimal for $r_0 \in [0.3, 0.7]$, with marginal improvements or compromises outside this range.

Deviating from $[0.3, 0.7]$ causes suboptimal performance due to excessive or insufficient exploration. Careful selection of r_0 ensures robust trade-offs.

2. **Impact of Inner-to-Outer Loop Learning Rate Ratio:** The learning rate ratio governs adaptation speed and stability. Ratios $1:1 \rightarrow 5:1$ were tested, the effects are summarized in Fig. 4.

   - For $1:1 \rightarrow 3:1$, $VR = 0.697$, $ART = 491.121$ s, $RUE = 0.758$, and $CE = 54.331$ indicate optimal performance.
   - Beyond $3:1$ ($4:1, 5:1$), degradation occurs: VR increases to 0.764 and 0.797, ART rises to 568.337 and 583.4 s, and RUE decreases to 0.707 and 0.688. Marginal improvements in CE (50.236, 48.157) compromise other metrics.

   Maintaining the ratio within $[1:1, 3:1]$ ensures convergence and stability, avoiding degradation caused by unbalanced updates.

These findings emphasize the importance of carefully tuning r_0 and the learning rate ratio for optimal performance in LFRL-MOS.

Table 5 Inference Time of Each Component and Total Runtime (Seconds)

Data Scale [1]	LLM	MOALO	RL	Total
100,20	9.8302	1.7421	0.0079	11.5802
1000,200	11.9017	32.7806	0.1094	44.7917
5000,1000	10.2032	105.4167	0.6633	116.2832
1000,2000	8.6511	423.2506	1.5594	433.4611
50000,5000	9.6552	1118.1450	28.4625	1156.2627

[1] The number of (Patients, Resources)

5.5.4 RQ4: Time Efficiency

To evaluate the computational efficiency of the LFRL-MOS framework, the paper measure the inference time of each core component (LLM, MOALO, RL) and the total runtime under varying task scales. The results are summarized in Table 5, where the time is reported in seconds.

From the table, we make the following observations:

1. **LLM Inference Time:** The LLM component maintains a stable inference time (around 9–12 seconds) across all data scales. This is primarily attributed to the prompt-based, zero-shot inference strategy and the efficient caching mechanism, which avoids redundant API calls for similar states. This suggests that LLM inference does not scale linearly with task size, making it computationally feasible even for large-scale applications.
2. **MOALO Optimization Time:** As expected, MOALO is the dominant time-consuming component, especially for larger data scales. For 50,000 patients and 5,000 resources, MOALO alone takes over 18 minutes, which reflects the high computational cost of evolutionary optimization. However, this aligns with the nature of population-based metaheuristics, which are inherently resource-intensive but effective for multi-objective decision-making.
3. **PPO Reinforcement Learning Time:** The PPO component contributes negligibly to the total runtime for small to medium data scales. However, with 50,000 tasks, its runtime increases to 28.46 seconds, reflecting the growing complexity in adaptive parameter tuning across multiple objectives.
4. **Total Runtime and Scalability:** The total runtime increases significantly with data size, primarily driven by the MOALO component. For 10,000 patients, the total runtime is about 7 minutes, which is acceptable for semi-offline or daily batch hospital scheduling. For 50,000 patients, the total runtime is around 19 minutes, which may be acceptable in centralized hospital systems where scheduling occurs in batches, but might be limiting for real-time deployment.

6 Conclusion

This paper proposes the LFRL-MOS framework to address resource scheduling challenges in dynamic environments. It integrates three key components: (1) LLM-enhanced fuzzy state fusion for unified real-time and predictive state representation; (2) SLA-constrained MOALO for multi-objective optimization with dynamic adjustments; (3) Dual-loop PPO for hierarchical parameter adaptation.

Experiments on a medical resource scheduling scenario demonstrate LFRL-MOS's effectiveness in reducing SLA violations, minimizing response times, maximizing resource utilization, and ensuring cost-effectiveness. Compared to state-of-the-art baselines such as NSGA-II, FACO, and CEGA-DRL, our framework exhibits superior performance under varying load conditions and SLA requirements. These results underscore the importance of synergizing advanced machine learning techniques with evolutionary optimization for tackling complex, multi-objective scheduling problems.

Future work will extend the framework by incorporating real-world hospital operational logs to validate its performance in practical clinical environments. Additionally, integrating the framework with distributed computing platforms (e.g., Apache Spark, Ray) could further enhance its scalability and runtime efficiency.

A The prompt information for LLM optimizing fuzzy control parameters

```
As a healthcare resource scheduling AI expert, please
optimize the fuzzy control parameters
 based on the following resource status:
== Current Status ==
1. Real-Time Cost: {real cost:.2f}
2. Real-Time Quality: {real quality:.2f}
3. Real-Time Resource Utilization: {real utilization:.1%}
4. Predictive Cost: {pred cost:.2f}
5. Predictive Risk: {pred risk:.1%}
6. Predictive Resource Utilization: {pred utilization:.1%}
== Parameter Description ==
''real_cost_mu_params": {
    ''mean_c": Real-Time Cost Mean
    ''std_dev_sigma": Real-Time Cost Standard
        Deviation
},
''real_quality_mu_params": {
    ''threshold_theta1": Real-Time Quality Threshold 1
    ''threshold_theta2": Real-Time Quality Threshold 2
    ''membership_values": Real-Time Quality
        Membership Values
},
''pred_cost_mu_params": {
    ''predicted_mean_c": Predictive Cost Mean
    ''predicted_std_dev_sigma": Predictive Cost
        Standard Deviation
},
''pred_risk_mu_params": {
    ''exponential_decay_k": Predictive Risk Exponential
        Decay Coefficient
},
''pred_utilization_mu_params": {
    ''predicted_utilization_u": Predictive Resource
        Utilization Mean
    ''dynamic_error_bound_w": Predictive Resource
        Utilization Error Bound
}
== Output Requirements ==
Provide the output in strict JSON format, including
optimized parameters and adjustment rationale
```

B The pseudocode process of MOALO algorithm

Algorithm 1 MOALO Algorithm

Require:
Patients $\mathcal{I}$, resources $\mathcal{J}$, departments $\mathcal{K}$
Time slots $\mathcal{T}$, max iterations T
SLA penalty sla_p, base parameters Θ
Ensure:
Pareto frontier $\mathcal{PF}$
Recommended schedule X^*
$P \leftarrow \text{InitializePopulation}(N, |\mathcal{I}|, |\mathcal{J}|, |\mathcal{K}|, |\mathcal{T}|)$
$A \leftarrow \emptyset$
for $t = 1$ **to** T **do**
  $P' \leftarrow \emptyset$
  for each X_i **in** P **do**
    $R_i \leftarrow \text{ComputeRadius}(t, X_i, P, sla_p)$
    $\tilde{X}_i \leftarrow X_i + R_i \times \mathcal{N}(0, \Sigma_i)$
    $\tilde{X}_i \leftarrow \text{RepairConstraints}(\tilde{X}_i)$
    $P' \leftarrow P' \cup \{\tilde{X}_i\}$
  end for
  $Q \leftarrow P \cup P'$
  $[\mathbf{f}, \phi] \leftarrow \text{EvaluateObjectives}(Q)$
  $F \leftarrow \text{FastNonDominatedSort}(Q, \phi)$
  $P \leftarrow \text{EliteSelection}(F, \eta(t))$
end for
$PF \leftarrow A$
$X^* \leftarrow \text{Selection}(A, \mathbf{w})$
return PF, X^*

Author Contributions Author 1: Methodology, Software, Writing. Author 2: Resources, Funding acquisition. Author 3: Supervision, Writing - review & editing. Author 4: Resources, Funding acquisition. Author 5: Resources, Funding acquisition.

Funding Research in this study is supported by the National Key Research and Development Program of China (No 2022YFF0902703), the Special Funding Program of Shandong Taishan Scholars Project, the Key Research and Development Program of Shandong Province (Grant 2020CXGC010903) and National Natural Science Foundation of China (Grant 62073103).

Data Availability The data that support the findings of this study are available from the corresponding author, upon reasonable request.

Declarations

Conflicts of Interest The authors declare no competing interests.

References

1. Pinedo M, Zacharias C, Zhu N (2015) Scheduling in the service industries: an overview. J Syst Sci Syst Eng 24(1):1–48. https://doi.org/10.1007/s11518-015-5266-0
2. Gupta S, Singh RS (2024) User-defined weight based multi objective task scheduling in cloud using whale optimization algorithm. Simul Model Pract Theory 133:102915. https://doi.org/10.1016/j.simpat.2024.102915
3. Joshi S, Pandey NK, Diwakar M, Mishra AK (2024) Reinforcement learning-driven auto scaling for sla enhancement in cloud environment. In: 2024 Eighth International Conference on Parallel, Distributed and Grid Computing (PDGC), pp 634–639. https://doi.org/10.1109/PDGC64653.2024.10984189. IEEE
4. Katoch S, Chauhan SS, Kumar V (2021) A review on genetic algorithm: past, present, and future. Multimed Tools and Appl 80:8091–8126. https://doi.org/10.1007/s11042-020-10139-6
5. Song Y, Wu Y, Guo Y, Yan R, Suganthan PN, Zhang Y, Pedrycz W, Das S, Mallipeddi R, Ajani OS et al (2024) Reinforcement learning-assisted evolutionary algorithm: a survey and research opportunities. Swarm Evol Comput 86:101517. https://doi.org/10.1016/j.swevo.2024.101517
6. Belboul Z, Toual B, Kouzou A, Bensalem A (2022) Optimization of hybrid pv/wind/battery/dg microgrid using moalo: a case study in djelfa, algeria. In: 2022 19th International Multi-Conference on Systems, Signals & Devices (SSD), pp 1615–1621. doi:https://doi.org/10.1109/SSD54932.2022.9955892. IEEE
7. Abraham OL, Ngadi MA, Sharif JBM, Sidik MKM (2025) Multi-objective optimization techniques in cloud task scheduling: a systematic literature review. IEEE Access 13:12255–12291. https://doi.org/10.1109/ACCESS.2025.3529839
8. Zhang Y, Sun L, Ma Z, Wang J, Fu M, Joung J (2025) A 5g-tsn joint resource scheduling algorithm based on optimized deep reinforcement learning model for industrial networks. Ad Hoc Netw 170:103783. https://doi.org/10.1016/j.adhoc.2025.103783
9. Xue F, Chen Y, Dong T, Wang P, Fan W (2025) Moea/d with adaptive weight vector adjustment and parameter selection based on q-learning. Appl Intell 55(6):399. https://doi.org/10.1007/s10489-024-05906-z
10. Liu Y, Pang K-W, Jin Y, Wang S, Zhen L (2025) Optimizing vessel scheduling in ports: an integer programming approach to mitigating extreme weather impacts. Comput Ind Eng 111134. https://doi.org/10.1016/j.cie.2025.111134
11. Wu Y, Tanaka S (2025) Perishable inventory control with backlogging penalties: a mixed-integer linear programming model via two-step approximation. Comput Oper Res 176:106953. https://doi.org/10.1016/j.cor.2024.106953
12. Chen X, Li J, Wang Z, Li J, Gao K (2024) A genetic programming based cooperative evolutionary algorithm for flexible job shop with crane transportation and setup times. Appl Soft Comput 169:112614. https://doi.org/10.1016/j.asoc.2024.112614
13. Teng G (2025) An improved genetic algorithm for dual-resource constrained flexible job shop scheduling problem with tool-switching dependent setup time. Expert Syst Appl 281:127496. https://doi.org/10.1016/j.eswa.2025.127496
14. Shao S, Xu G, Li J, Liu Z, Jin Z (2025) A job assignment scheduling algorithm with variable sublots for lot-streaming flexible job shop problem based on nsgaii. Comput Oper Res 173:106866. https://doi.org/10.1016/j.cor.2024.106866
15. Pugliese V, Ferreira O, Faria F (2025) Hybrid flow shop scheduling through reinforcement learning: a systematic literature review. In: Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, pp 1240–1249. https://doi.org/10.1145/3672608.3707903
16. Hou Y, Liao X, Chen G, Chen Y (2025) Co-evolutionary nsga-iii with deep reinforcement learning for multi-objective distributed flexible job shop scheduling. Comput Ind Eng 203:110990. https://doi.org/10.1016/j.cie.2025.110990
17. Kshatriya D, Lepakshi VA (2024) Sla aware optimized task scheduling model for faster execution of workloads among federated clouds. Wireless Pers Commun 135(3):1635–1661. https://doi.org/10.1007/s11277-024-11135-x
18. Samimi A, Goudarzi M (2025) A novel strategy for optimal data replication in peer-to-peer networks based on a multi-objective optimisation-nsga-ii algorithm. IET Netw 14(1):12141. https://doi.org/10.1049/ntw2.12141
19. Lipsa S, Dash RK (2024) Sla-based task scheduling in cloud computing using randomized pso algorithm. In: SCI, pp. 206–217. doi:https://doi.org/10.1049/ntw2.12141
20. Sargolzaei S, Mishmast Nehi H (2024) Multi-objective interval type-2 fuzzy linear programming problem with vagueness in coefficient. CAAI Trans on Intell Technol 9(5):1229–1248. https://doi.org/10.1049/cit2.12336
21. Maji S, Maity S, Giri D, Nielsen I, Maiti M (2025) Multi-objective multi-path covid-19 medical waste collection problem with type-2 fuzzy logic based risk using partial opposition-based weighted genetic algorithm. Eng Appl Artif Intell 143:109916. https://doi.org/10.1016/j.engappai.2024.109916
22. Manjotho AA, Tewolde TT, Duma RA, Niu Z (2025) Llm-guided fuzzy kinematic modeling for resolving kinematic uncertainties and linguistic ambiguities in text-to-motion generation. Expert Syst Appl 279:127283. https://doi.org/10.1016/j.eswa.2025.127283
23. Guo D, Yang D, Zhang H, Song J, Zhang R, Xu R, Zhu Q, Ma S, Wang P, Bi X et al. (2025) Deepseek-r1: incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948
24. Li Z, Piao C, Chu D, Tu Z, Hu X, Ding D (2025) Resource state adaptive collaboration mechanism based on resource modeling and multi-agent system. Complex Intell Syst 11(6):1–33. https://doi.org/10.1007/s40747-025-01882-0
25. Armony M, Israelit S, Mandelbaum A, Marmor YN, Tseytlin Y, Yom-Tov GB (2015) On patient flow in hospitals: a data-based queueing-science perspective. Stochastic Syst 5(1):146–194. https://doi.org/10.1287/14-SSY153
26. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: nsga-ii. IEEE Trans Evol Comput 6(2):182–197. https://doi.org/10.1109/4235.996017
27. Opris A, Dang D-C, Neumann F, Sudholt D (2024) Runtime analyses of nsga-iii on many-objective problems. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 1596–1604. https://doi.org/10.1145/3638529.3654218
28. Mao J (2024) Optimizing enterprise marketing project portfolios using fuzzy ant colony optimization. Int J of Comput Commun Control 19(3). https://doi.org/10.15837/ijccc.2024.3.6458
29. Jain R, Jain P, Tyagi B (2025) A model for sla aware admission control and qos aware task scheduling in cloud environment. Peer-to-Peer Netw and Appl 18(4):197. https://doi.org/10.1007/s12083-025-01988-9